NALYTICS
OJECT

Project scope, architecture, development roadmap, folder
and practical testing datasets

Project goal

Extend an existing Retrieval-Augmented Generation (RAG) application into a multi-tool AI agent. The agent will use custom Pandas MCP tools for structured analysis, a RAG MCP server for contextual retrieval, and a Web Search MCP server for external information. A central MCP Aggregator will expose the combined tools to a LangChain-powered LLM agent, with Flask providing the user interface.

Recommended learning sequence: build the Pandas MCP server first, integrate the remote web search server, reuse the existing RAG server, aggregate the servers, connect the LLM agent, and finally integrate Flask.

Project guide • Python • FastMCP • MCP • Pandas • LangChain • Flask

Contents and project overview

- 1. Project overview and learning objectives
- 2. Target architecture and transport choices
- 3. Scope and module descriptions
- 4. Testing datasets and exercises
- 5. Expected AI agent workflows
- 6. Recommended project folder structure
- 7. Required configuration files
- 8. README.md starter content
- 9. Development sequence and deliverables
- 10. Assessment criteria and implementation notes

1. Project overview and learning objectives

The project develops an AI-powered data analytics application that accepts an uploaded dataset and answers natural-language questions by selecting and combining tools exposed through MCP.

Learning objectives

- Understand MCP clients, servers, tools and transport mechanisms.
- Create custom MCP servers using FastMCP and expose local tools using STDIO.
- Build reusable Python analytics functions with Pandas.
- Connect to a remote Web Search MCP server using its supported HTTP transport.
- Reuse an existing RAG MCP server and understand the difference between retrieval and calculation.
- Build an MCP Aggregator that connects multiple MCP servers and exposes their tools through one endpoint.
- Connect the aggregator to an LLM agent using LangChain.
- Integrate the agent into a Flask application with dataset upload and chat functionality.

Functional scope

Component	Responsibilities
Pandas MCP Server	Dataset information, descriptive statistics, aggregation, GroupBy, filtering and validation.
RAG MCP Server	Index uploaded content and retrieve relevant records or passages.
Web Search MCP Server	Search the internet and retrieve external information or snippets.
MCP Aggregator	Connect to local and remote MCP servers and expose their combined tools.
LangChain AI Agent	Interpret questions, select tools, execute workflows and compose answers.
Flask Application	Upload files, manage user interaction and display responses.

2. Target architecture and transport choices

The Flask application communicates with the LangChain agent. The agent connects to the MCP Aggregator through a single MCP endpoint. The aggregator manages connections to the individual servers.



Pandas MCP	RAG MCP	Web Search MCP
STDIO Custom local server Aggregation & GroupBy	STDIO Existing local server Semantic retrieval	Streamable HTTP Remote service External search

Transport design

Server	Suggested transport	Reason
Pandas MCP	STDIO	A locally implemented Python process launched by the aggregator.
RAG MCP	STDIO	Reuse the local RAG implementation as a subprocess.
Web Search MCP	Streamable HTTP (or provider-supported transport)	Connect to an external MCP service.
MCP Aggregator	Streamable HTTP	Expose the combined tool set to the AI agent.

Transport clarification: STDIO servers are normally launched as subprocesses and communicate over standard input/output. Remote MCP servers commonly use Streamable HTTP. Confirm the transport and endpoint supported by the selected Web Search MCP provider; do not assume every provider uses the same URL or authentication.

3. Scope and module descriptions

Module 1 — Develop the Pandas MCP Server

Build reusable Pandas functions and expose them as MCP tools using FastMCP. The server should load or access the currently selected dataset, validate input parameters and return JSON-serializable results.

Tool name	Purpose	Example
get_dataset_info	Return dimensions, columns and data types.	How many rows and columns are present?
get_column_summary	Return descriptive statistics for a column.	What is the average passenger age?
aggregate_data	Apply count, sum, mean, min or max.	How many passengers survived?
groupby_data	Group by one or more columns and aggregate measures.	Average fare by passenger class.
filter_dataset	Filter records using validated conditions.	Passengers older than 50 who survived.

Implementation requirements

- Use FastMCP and STDIO transport for the local Pandas server.
- Separate ordinary analytics functions into analytics.py and MCP registration into pandas_server.py.

- Validate column names, aggregation functions and filter operators.
- Handle missing values and empty results explicitly.
- Return results in a format that can be serialized to JSON.
- Avoid arbitrary Python or SQL execution from LLM-supplied strings.

Illustrative Pandas operations

```
df.groupby("Pclass")["Fare"].mean()
```

```
df.groupby("Sex").agg({\n    "Survived": "mean",\n    "Age": "mean",\n    "Fare": "mean"\n})
```

Module 2 — Integrate Web Search MCP

Connect an existing Web Search MCP server rather than implementing a search engine. A provider such as Tavily may be used if its MCP server and account requirements are suitable. Configure the actual endpoint, authentication and transport according to provider documentation.

- Search the internet for external information.
- Retrieve relevant pages, snippets or search results.
- Research companies, industries, technologies and project-management practices.
- Use web results to add context to dataset findings, while distinguishing retrieved facts from calculations.

Module 3 — Integrate the existing RAG MCP Server

Reuse the current RAG project and expose its existing retrieval capabilities through MCP. Suggested tools:

Tool	Purpose
rag_load_dataset	Load and index the uploaded dataset or supported document.
rag_search_dataset	Retrieve relevant records or text based on a query.
rag_get_dataset_info	Return information about the indexed dataset.

RAG versus Pandas: RAG retrieves relevant content; Pandas calculates exact values from structured records. For example, use RAG to retrieve a description of a module, but use Pandas to count its tasks or calculate its average completion percentage.

Module 4 — Develop the MCP Aggregator

- Connect to the Pandas MCP server over STDIO.
- Connect to the RAG MCP server over STDIO.
- Connect to the Web Search MCP server over its supported remote transport.
- Expose the combined tools through one MCP endpoint, for example http://127.0.0.1:8000/mcp.
- Test that all expected tools are discoverable and callable through the aggregator.

Module 5 — Build the LangChain AI Agent

- Connect the agent to the MCP Aggregator.
- Retrieve the available MCP tools and provide them to the LLM.
- Interpret user questions and select the appropriate tools.
- Execute one or multiple tools when required.
- Combine tool outputs into a clear natural-language response.
- Handle tool errors, unavailable services and ambiguous questions.

Module 6 — Integrate with Flask

- Allow CSV and Excel uploads.
- Display dataset name, row count, columns and upload status.
- Provide a chat interface for natural-language questions.
- Pass the active dataset context to the analytics and RAG services.
- Display text answers and structured results such as tables.
- Validate uploads and isolate user sessions where multiple users are supported.

4. Testing datasets and exercises

Dataset A — Titanic passenger dataset

Use the public Titanic CSV dataset for introductory structured analytics. Suggested source: mwaskom/seaborn-data Titanic CSV on GitHub.

The dataset contains passenger information such as survival status, passenger class, sex, age, siblings/spouses, parents/children, fare, embarkation port, passenger category and whether the passenger travelled alone.

Column	Description
survived	Survival indicator (0 or 1).
pclass	Passenger class (1, 2 or 3).
sex	Passenger sex as represented in the dataset.
age	Passenger age.
sibsp	Number of siblings or spouses aboard.
parch	Number of parents or children aboard.
fare	Ticket fare.
embarked	Port of embarkation.
class	Passenger class label.
who	Passenger category, such as man, woman or child.
alone	Whether the passenger travelled alone.

Titanic practical exercises

#	Question	Expected tool
1	How many passengers are in the dataset?	Dataset Info
2	What is the average passenger age?	Column Summary
3	What is the average fare for each passenger class?	GroupBy
4	Calculate the survival rate by sex.	GroupBy
5	Calculate the number of survivors and non-survivors.	Aggregation / GroupBy
6	Which passenger class had the highest survival rate?	GroupBy
7	Compare average age between survivors and non-survivors.	GroupBy
8	How many passengers travelled alone?	Aggregation / Filter
9	Filter passengers older than 50 who survived.	Filter
10	Which embarkation port had the highest survival rate?	GroupBy

Titanic advanced multi-tool exercises

1. Calculate survival rate by passenger class, then search the web for historical context about class differences.
2. Calculate average fares by passenger class and use web research to explain possible differences.
3. Retrieve relevant passenger information using RAG and calculate group statistics using Pandas.

Dataset B — SilverBank Non-Banking Extension project plan

Use the uploaded ProjectPlan.xlsx workbook as the business analytics exercise. The workbook includes a DevPlan worksheet and project planning fields. The project names visible in the workbook include **SIBS II Epic 3 Modularization** and **SilverBank Non-Banking Extension**. Use the SilverBank Non-Banking Extension plan for this exercise, and verify the relevant project rows before analysis.

Fields identified in the project plan include:

Field	Description
Type	Task or work item classification.
Module	Project module or functional area.
Construct	Development construct.
File or Program Code	Related program or file reference.
WBS	Work breakdown structure identifier.
Task	Development task description.
Dependency	Task dependency.
Status	Task status.
Assigned To	Assigned developer or owner.
Target Manday	Estimated development effort.
Completion %	Reported progress percentage.
Overdue	Overdue indicator.
Duration	Task duration.
Planned / Actual Dates	Planned and actual start/end dates.
Remarks	Additional notes.
Financial / Non-Financial Mapping	Business classification.

Examples of development areas represented in the workbook include General Administration, Accounts Payable and Receivable, Taxation and Documentation, Debt Management, UI Generator, API Service Program, Banking Events and Counterparty Registration.

SilverBank practical exercises

#	Question	Expected tool
1	How many tasks are in the project plan?	Aggregation
2	How many tasks are completed, pending and in progress?	GroupBy
3	How many tasks are assigned to each developer?	GroupBy
4	Which module has the highest number of tasks?	GroupBy
5	Calculate average completion percentage by module.	GroupBy
6	Which tasks have not been assigned?	Filter
7	How many tasks are Financial and Non-Financial?	GroupBy
8	Identify tasks marked as overdue.	Filter
9	Calculate total estimated mandays by module.	Aggregation
10	Which developers have the highest number of assigned tasks?	GroupBy

SilverBank advanced multi-tool exercises

1. Summarise project progress and identify modules with the lowest completion percentage.
2. Identify unassigned development tasks and suggest a review of workload allocation.
3. Compare planned and actual dates to identify possible schedule delays.
4. Analyse low-completion areas and search for best practices to improve software project delivery.
5. Use RAG to explain the Debt Management module and Pandas to count related tasks.
6. Identify modules with the highest estimated effort and explain their relevance using retrieved project context.

Data handling note: Before distributing the project workbook, review and anonymise developer names, internal project information and confidential business details. Do not assume a blank completion field means zero; define and document how blanks are handled. Confirm that task filters and project identifiers select only the intended SilverBank project rows.

5. Expected AI agent workflows

The key learning outcome is intelligent tool selection and orchestration—not simply making each MCP server work independently.

Example 1 — Titanic data analysis

User question: Calculate the average fare for each passenger class and tell me which class paid the most.

1. The LLM identifies this as a structured calculation.
2. The agent calls the Pandas MCP groupby tool using passenger class and average fare.
3. The tool returns the grouped results.
4. The LLM explains which class has the highest average fare.

Illustrative output (not a live execution of the supplied CSV):

Passenger class	Illustrative average fare
First	84.15
Second	20.66
Third	13.68

Example 2 — Project workload analysis

User question: Which developers have the highest number of tasks, and are there any unassigned tasks?

1. Pandas GroupBy counts tasks by assigned developer.
2. Pandas Filter identifies missing or blank assignments.
3. The LLM summarises workload distribution and flags unassigned work.

Example 3 — Data plus web search

User question: Analyse the project tasks with the lowest completion rates and search for best practices to improve delivery.

1. Pandas identifies modules or tasks with low completion.
2. Web Search MCP finds relevant project delivery practices.
3. The LLM combines the measured findings with external research and clearly distinguishes the two.

Example 4 — RAG plus data analysis

User question: Explain the Debt Management module and calculate how many related development tasks are assigned.

1. RAG MCP retrieves relevant descriptions and project context.
2. Pandas MCP filters the project plan and counts related tasks.
3. The LLM combines the retrieved description and calculated count.

Expected agent behaviour checklist

- Use Pandas for exact numeric calculations over structured data.
- Use RAG for semantic retrieval from indexed content.
- Use Web Search for external information, not as a substitute for uploaded dataset calculations.
- Call multiple tools when the question requires multiple sources.

- State assumptions, missing data and limitations.
- Do not invent results when a tool fails or returns no data.

6. Recommended project folder structure

```
MCP_Data_Analytics_Project/
|
|-- app.py
|-- requirements.txt
|-- .env
|-- .env.example
|-- .gitignore
|-- README.md
|
|-- config/
|   |-- settings.py
|   |-- prompts.py
|
|-- data/
|   |-- titanic.csv
|   |-- ProjectPlan.xlsx
|
|-- uploads/
|
|-- pandas_mcp/
|   |-- __init__.py
|   |-- pandas_server.py
|   |-- analytics.py
|
|-- rag_mcp/
|   |-- __init__.py
|   |-- rag_server.py
|   |-- embeddings.py
|   |-- vector_store.py
|
|-- web_search_mcp/
|   |-- __init__.py
|   |-- web_search_config.py
|
|-- aggregator/
|   |-- __init__.py
|   |-- aggregator_server.py
|
|-- agent/
|   |-- __init__.py
|   |-- mcp_client.py
|   |-- ai_agent.py
|
|-- templates/
|   |-- index.html
|
|-- static/
|   |-- css/style.css
|   |-- js/script.js
|
|-- logs/
    |-- app.log
```

Path	Responsibility
app.py	Main Flask application.
requirements.txt	Python dependencies.
.env / .env.example	Secrets and configuration / safe template.
README.md	Installation, architecture, usage and testing instructions.
config/	Settings and system prompts.
data/ and uploads/	Sample datasets and uploaded files.
pandas_mcp/	Reusable Pandas functions and FastMCP server.

Path	Responsibility
rag_mcp/	Existing RAG MCP implementation.
web_search_mcp/	Remote web search connection configuration.
aggregator/	MCP server connections and combined endpoint.
agent/	MCP client and LangChain agent.
templates/ and static/	Flask UI, CSS and JavaScript.
logs/	Application logs for debugging.

Keep each responsibility separate: analytics.py contains ordinary Python functions; pandas_server.py registers them as MCP tools; aggregator_server.py manages MCP server connections; mcp_client.py connects the agent to the aggregator; ai_agent.py configures the LLM and tool-calling workflow; app.py serves Flask.

7. Required configuration files

A. requirements.txt — suggested starting dependencies

```
# Flask application
flask

# Data analytics
pandas
numpy
openpyxl

# MCP framework
fastmcp

# LangChain and MCP integration
langchain
langchain-core
langchain-openai
langchain-mcp-adapters

# LLM API client
openai

# RAG and vector search
sentence-transformers
faiss-cpu

# Environment configuration
python-dotenv

# Utilities
requests
```

This is a starting dependency list, not a tested lockfile. Install in a virtual environment, resolve compatible versions, run the project tests, and pin the working versions before sharing the project.

B. .env — local environment configuration

```
# LLM
OPENROUTER_API_KEY=your_openrouter_api_key
MODEL=your_model_name

# Web Search MCP
WEB_SEARCH_MCP_URL=your_web_search_mcp_endpoint
TAVILY_API_KEY=your_tavily_api_key

# MCP Aggregator
AGGREGATOR_HOST=127.0.0.1
AGGREGATOR_PORT=8000

# Flask
FLASK_HOST=127.0.0.1
FLASK_PORT=5000
FLASK_DEBUG=True
SECRET_KEY=replace_with_a_random_secret

# RAG
EMBEDDING_MODEL=sentence-transformers/all-MiniLM-L6-v2
VECTOR_STORE_PATH=./vector_store

# Data
UPLOAD_FOLDER=./uploads
```

Use only the credentials required by the selected provider. A Web Search MCP may require an API key, OAuth or another authentication mechanism. Configure it according to its documentation. Never commit the real .env file to version control.

C. .env.example — shareable template

```
OPENROUTER_API_KEY=
MODEL=

WEB_SEARCH_MCP_URL=
```

```
TAVILY_API_KEY=

AGGREGATOR_HOST=127.0.0.1
AGGREGATOR_PORT=8000

FLASK_HOST=127.0.0.1
FLASK_PORT=5000
FLASK_DEBUG=True
SECRET_KEY=

EMBEDDING_MODEL=sentence-transformers/all-MiniLM-L6-v2
VECTOR_STORE_PATH=./vector_store

UPLOAD_FOLDER=./uploads
```

D. .gitignore

```
.venv/
venv/
__pycache__/
*.pyc

.env

uploads/*
!uploads/.gitkeep

vector_store/
logs/*.log

.ipynb_checkpoints/
.DS_Store
```

Security and configuration reminders

- Use a strong random Flask secret key and keep it private.
- Do not commit API keys, passwords or private project datasets.
- Validate file extensions, file sizes and content before processing uploads.
- Do not expose the development Flask server or MCP endpoint publicly without appropriate security controls.
- For multi-user deployments, isolate uploaded files, vector indexes and dataset state by user/session.
- Use supported, documented tool parameters and validate all LLM-provided inputs.

8. README.md starter content

Participants should create a README.md in the project root. The following is a starter template to customise after the implementation is complete.

```
# MCP-Based Data Analytics AI Agent

## 1. Project Overview
An AI-powered data analytics application using MCP.
Users upload datasets and ask natural-language questions.
The agent can use Pandas MCP, RAG MCP and Web Search MCP.

## 2. Architecture
Flask Application
    |
LangChain AI Agent
    |
MCP Aggregator
    |-- Pandas MCP (STDIO)
    |-- RAG MCP (STDIO)
    |-- Web Search MCP (HTTP)

## 3. Technologies
- Python
- Flask
- Pandas
- FastMCP
```

```
- Model Context Protocol
- LangChain
- OpenRouter (or configured LLM provider)
- FAISS
- Sentence Transformers

## 4. Features
### Pandas MCP
- Dataset information
- Descriptive statistics
- Aggregation
- GroupBy
- Filtering

### RAG MCP
- Dataset indexing
- Semantic search
- Contextual retrieval

### Web Search MCP
- Internet search
- External information retrieval

### AI Agent
- Natural-language interaction
- MCP tool selection
- Multi-tool workflows
- Response generation

## 5. Installation
1. Clone or copy the project.
2. Create and activate a Python virtual environment.
3. Install dependencies: pip install -r requirements.txt
4. Copy .env.example to .env.
5. Add the required LLM and Web Search credentials.
6. Place sample datasets in data/.

## 6. Running
1. Start the MCP Aggregator using the project's
   configured launch command.
2. Confirm that the aggregator can discover all tools.
3. Start Flask: python app.py
4. Open http://127.0.0.1:5000
5. Upload a supported dataset and ask a question.

## 7. Testing
Titanic:
- Survival rate by sex and passenger class
- Average fare by passenger class
- Passenger counts and age summaries

SilverBank project plan:
- Task counts by status and module
- Developer workload
- Unassigned and overdue tasks
- Completion percentage and estimated mandays

## 8. Expected Outcome
A working AI agent that combines structured data
analysis, contextual retrieval and external web search.

## 9. Security
- Keep credentials in .env.
- Never commit secrets.
- Validate uploaded files.
- Protect private project data.
- Apply access controls before deployment.
```

The README must be updated to match the actual commands, server ports, supported file formats, environment variables and provider configuration used by the completed implementation. Do not leave placeholder commands or credentials in the final submission.

9. Development sequence and deliverables

Recommended development sequence

Stage	Development activities	Completion evidence
1. Pandas MCP	Create analytics.py; implement dataset info, summary, aggregation, GroupBy and filter; register tools in FastMCP; use STDIO.	Tools callable independently through an MCP client.
2. Web Search MCP	Choose a compatible provider; configure credentials and remote connection; test search.	Search tool returns usable results.
3. RAG MCP	Reuse the existing RAG server; connect through STDIO; test loading and retrieval.	RAG tools discoverable and callable.
4. Aggregator	Connect all three MCP servers; expose combined tools over Streamable HTTP.	All expected tools visible through one endpoint.
5. LangChain agent	Connect to aggregator; configure LLM; test tool selection and multi-tool workflows.	Agent chooses and executes appropriate tools.
6. Flask integration	Implement upload, dataset state, chat and result display.	User can upload and ask questions through browser.
7. Final testing	Test both datasets; handle errors; document setup and limitations.	Demonstration, test evidence and final documentation.

Final project deliverables

Deliverable	Requirement
Pandas MCP Server	Working local STDIO server with analytics tools.
RAG MCP Server	Existing RAG server integrated and tested.
Web Search MCP	Working remote MCP integration.
MCP Aggregator	Combines all three MCP servers.
MCP Client and AI Agent	Connects to aggregator and executes tools.
Flask Application	Dataset upload and chat interface.
Configuration	requirements.txt, .env, .env.example and .gitignore.
Documentation	README.md with setup, architecture and usage.
Testing Evidence	Screenshots, example prompts and observed outputs.

10. Suggested assessment criteria

Assessment area	Weight
Pandas MCP tool development	20%
Web Search and RAG integration	15%
MCP Aggregator implementation	20%
LangChain AI Agent integration	20%
Flask application	10%
Testing and documentation	15%
Total	100%

Implementation notes and design recommendations

- **Keep the Pandas server independent of Flask.** Test it with an MCP client or inspector before integrating it into the application.
- **Define a shared dataset lifecycle.** The Pandas and RAG servers need access to the same active upload. Use a shared dataset path or a common dataset manager, and ensure the correct dataset is selected for each user/session.
- **Keep the aggregator focused on orchestration.** It should manage MCP connections and expose tools, not perform dataset analytics or store uploaded files.
- **Use clear tool schemas.** Tool parameters should explicitly describe dataset identifiers, columns, aggregation functions, grouping columns, filters and output limits.
- **Handle multi-tool answers carefully.** The LLM should identify which results came from calculations, RAG retrieval and web search.
- **Test incrementally.** Verify each MCP server before integrating it into the aggregator, then verify the aggregator before connecting the LLM agent.

Final expected outcome

A functional web application in which a user can upload the Titanic dataset or the SilverBank project plan, ask questions in natural language, and receive answers supported by Pandas calculations, RAG retrieval and external web search. The agent should be able to select one tool or combine multiple tools depending on the question.

Core learning outcome: Participants learn to create custom MCP tools, expose local tools through STDIO, connect remote MCP services through HTTP, aggregate multiple servers and make the combined tools available to an LLM-powered AI agent.

Reference dataset: Titanic CSV — https://github.com/mwaskom/seaborn-data/blob/master/titanic.csv
Project workbook: ProjectPlan.xlsx supplied with the project brief. Review and sanitise before classroom distribution.